Session 3: Fine-Tuning in Practice

Adaptation Choices

ADAPTATION CHOICES: CPT vs SFT

- Continued Pre-Training (CPT) and Supervised Fine-Tuning (SFT) are both forms of fine-tuning

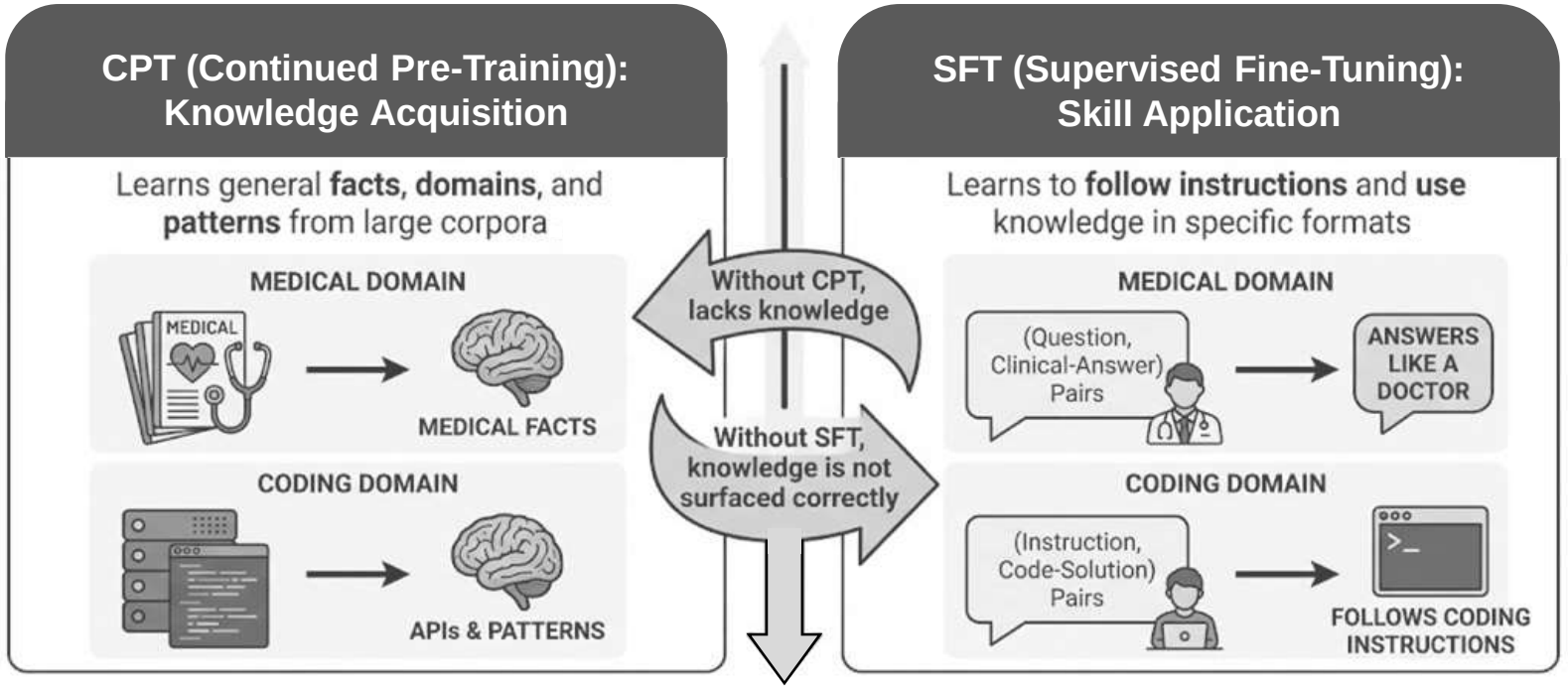
- They target fundamentally different things. Choosing the right method, or combining them, depends on what your model is missing

- What each method does

	CPT	SFT
Goal	Inject new **knowledge** (domain, facts, language)	Teach new **behaviour** (instruction-following, format, style)
Data	Large corpus of raw or lightly filtered text (domain books, internal docs, code repos)	Curated (instruction, response) pairs or conversations
Objective	Standard next-token prediction on the raw text	Next-token prediction on response tokens only (instruction is context)
What changes	The model adapts its internal representations to domain-specific vocabulary, facts, and patterns	The model learns how to use its knowledge in a user-facing way

CONCEPTUAL ANALOGY: CPT vs SFT

Think of CPT as **reading textbooks** and SFT as **practising exam questions**



The Synergy: Knowledge + Behaviour

CPT provides the **knowledge** base, SFT provides the **behaviour** to apply it effectively. Both are essential for a capable LLM

WHEN TO GO FOR CPT

The domain is underrepresented in pretraining data

- Medical, legal, financial, or internal-enterprise text that wasn't in the web crawl
- The base model may not have been exposed to these facts during pretraining

The vocabulary is specialised

- Domain-specific jargon, abbreviations, or even a different language that the tokeniser handles poorly
- CPT lets the model build better representations for these tokens

Need factual depth, not just style

- SFT can teach format and tone, but it can't teach knowledge the model never saw
- If the model hallucinates domain facts, CPT (with quality data) is the fix

There is enough domain text

- CPT needs a meaningful corpus (thousands to millions of tokens) to shift the model's knowledge distribution
- A few hundred examples won't do

WHEN SFT ALONE IS ENOUGH

01 The domain is already well-covered by the base model (e.g. general English Q&A)

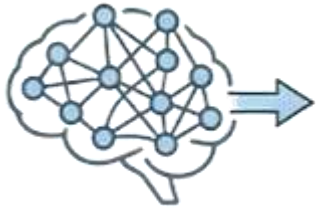
02 We mainly need to change format, tone, or instruction-following; not inject new facts

03 Our dataset is small and curated: consisting of (instruction, response) pairs rather than raw text

COMBINING CPT AND SFT

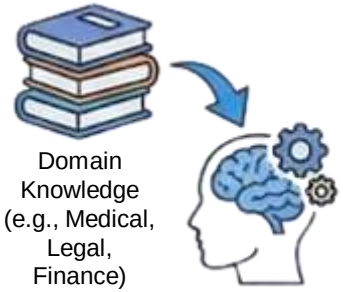
The most robust pipeline for a specialised product is often the following

BASE MODEL



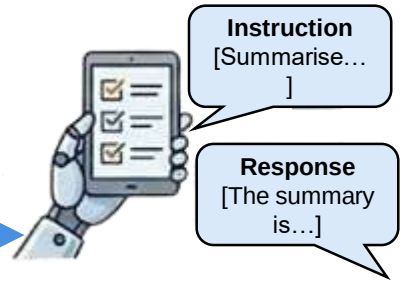
Foundation, pre-trained model

CPT (Domain Corpus)



Gives the model domain knowledge

SFT (Task-Specific Instruction Pairs)



Teaches it to use that knowledge in your product's format

Alignment

Refines safety and preferences

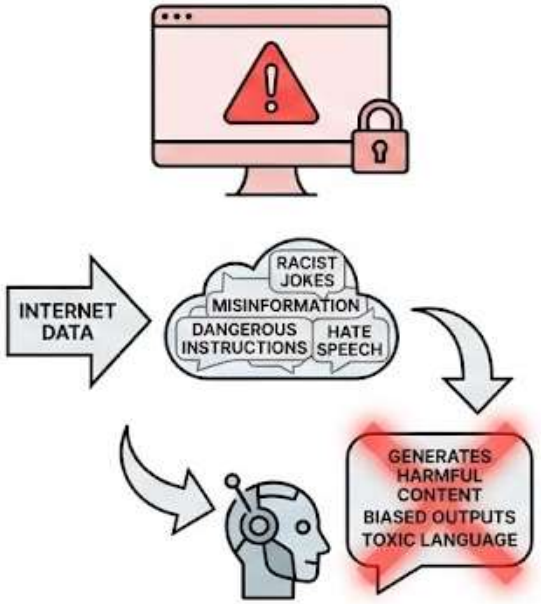
This pipeline ensures the model has the right knowledge, uses it correctly for the task, and behaves safely and helpfully in real-world applications

Alignment

WHY ALIGNMENT IS NEEDED

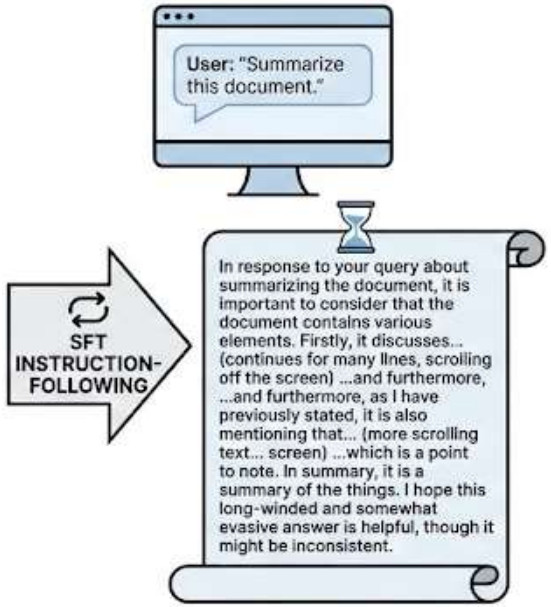
After pre-training (and even after SFT), a model can

Harmful/Bias/Toxic Generation (Pretraining Data)



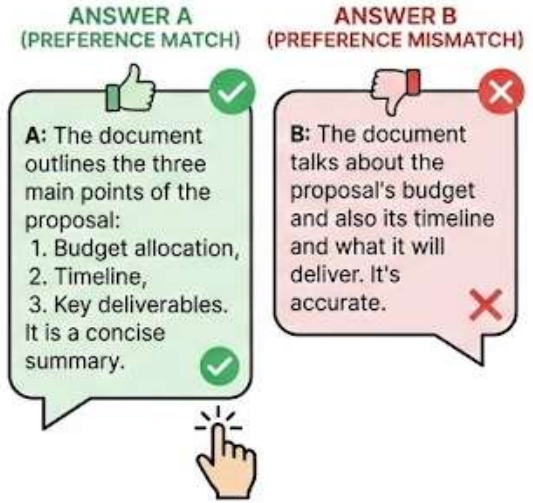
Unfiltered Internet → Model ingestion

Unhelpful/Verbose (SFT Limitation)



Follows instructions → But long-winded and evasive

Preference Mismatch (Beyond SFT)



Multiple responses may be correct → But SFT doesn't rank quality

Alignment bridges the gap between "can generate correct text" and "reliably generates text humans prefer and trust"

REINFORCEMENT LEARNING FROM HUMAN FEEDBACK

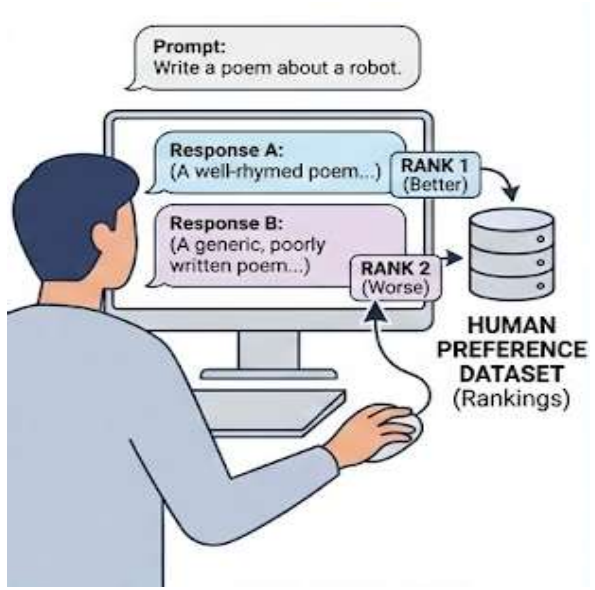
RLHF (Reinforcement Learning from Human Feedback) is one of the most widely used alignment methods. It sits after SFT in the pipeline

With RLHF, the model learns to generate outputs that humans prefer, not just outputs that are plausible next-token continuations

RLHF PROCESS

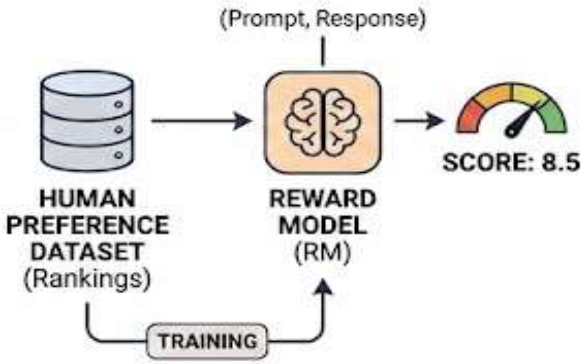
RLHF essentially follows a three-step process

Collect Preference Data



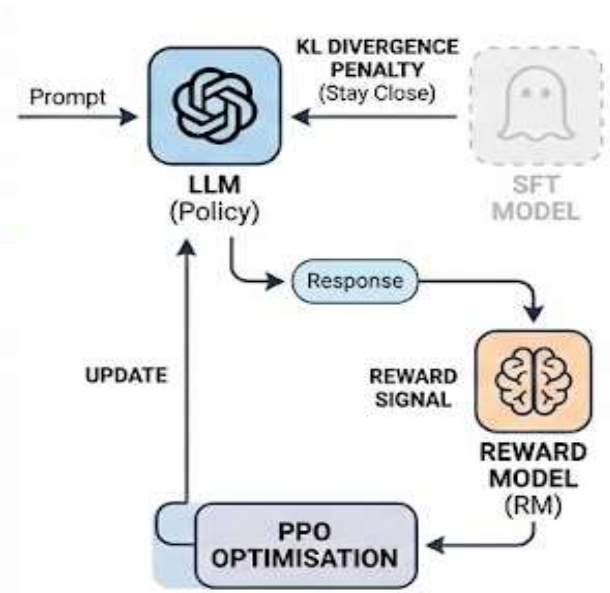
Show humans multiple responses to a prompt; they rank them

Train a Reward Model (RM)



Train a separate model on preference data to score any response

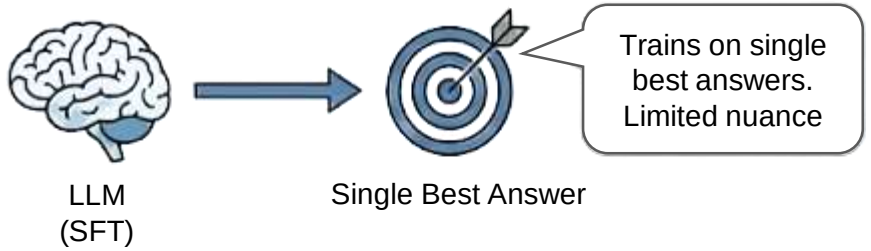
Optimise the Policy (LLM)



Use RL (PPO) to update LLM for high rewards, while staying close to SFT model

WHY RLHF AFTER SFT

SFT (Supervised Fine-Tuning)
Limitations



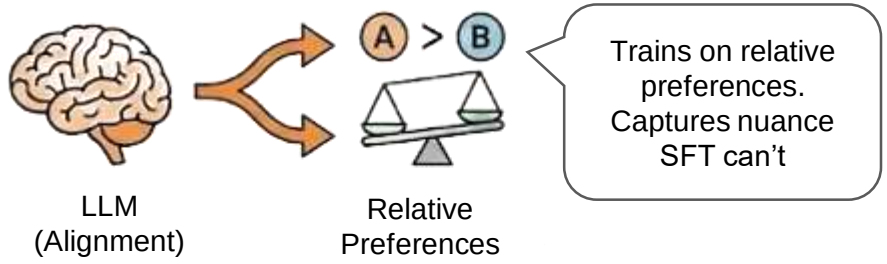
Can’t encode complex preferences

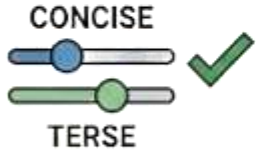
(e.g., “Concise but not Terse”, “Avoid Style”)

Risk of reward hacking

(Drifting too far from base policy, no explicit penalty)

Alignment (RLHF/Preference Learning)
Advantages





Expresses trade-offs naturally

(e.g., “Concise > Terse”, “Style A > Style B”)

Explicitly penalises reward hacking

(Stays close to base policy)

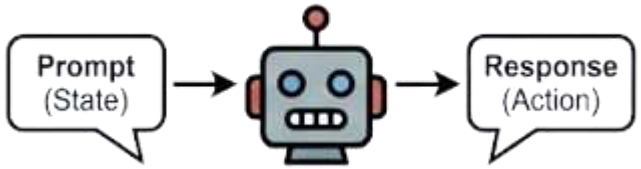
Alignment captures relative quality and trade-offs, prevents reward hacking, and enables finer control, which SFT alone cannot achieve

Reinforcement Learning Methods

PROXIMAL POLICY OPTIMISATION

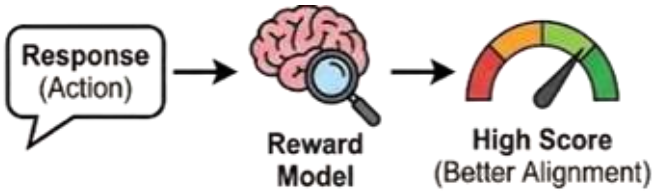
Proximal Policy Optimisation (PPO) is the RL algorithm behind classic RLHF. PPO nudges the model toward responses the reward model likes, one small step at a time, without drifting too far from the original model

1. LLM is Policy



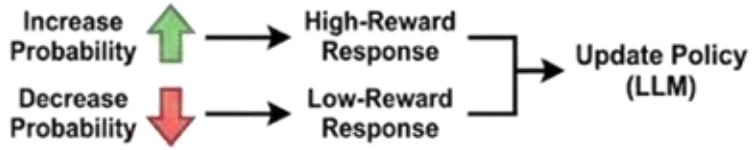
The LLM is a "policy": given a prompt (state), it generates a response (action)

2. Reward Model and Scoring



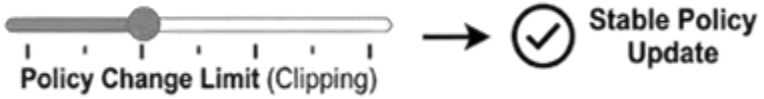
The reward model scores the response: higher score = better alignment with human preferences

3. PPO Update



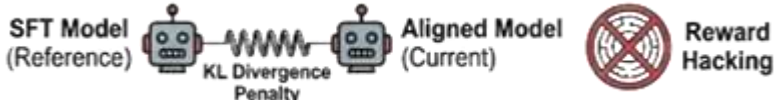
PPO updates the policy to increase the probability of high-reward responses and decrease the probability of low-reward ones

4. "Proximal" Constraint (Clipping)



PPO limits how the policy can change in one step (via a clipping mechanism). This prevents the model from making large, destabilising jumps

5. KL Penalty (Prevent Reward Hacking)



An additional term keeps the aligned model close to the SFT model; preventing "reward hacking", where the model exploits quirks of the reward model instead of genuinely improving

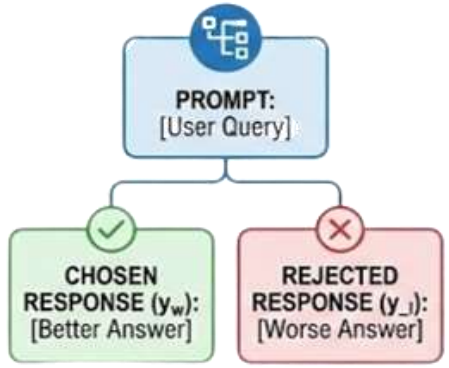
PPO CHALLENGES

Challenge	Why it's hard
Reward model quality	The RM is itself an approximation of human preferences; if it's noisy or biased, PPO optimises for the wrong thing
Reward hacking	The model can learn to exploit RM blind spots (e.g. verbosity that scores high but isn't truly better)
Training instability	RL training is inherently less stable than supervised learning; hyperparameter sensitivity, reward collapse, and mode collapse are common
Infrastructure complexity	PPO needs four models in memory simultaneously: the policy, the reference (SFT) model, the reward model, and the value model. This is expensive and complex to orchestrate
Sample efficiency	PPO generates responses, scores them, and updates; loop is slow compared to supervised updates

DIRECT PREFERENCE OPTIMISATION

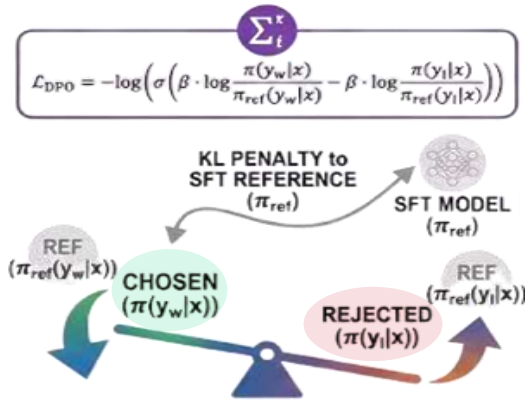
- Direct Preference Optimisation (DPO) achieves alignment without training a separate reward model or running RL
- DPO shows that the optimal RL solution (under certain assumptions) can be expressed as a supervised loss directly on the preference pairs

Input: Preference Data Triplets



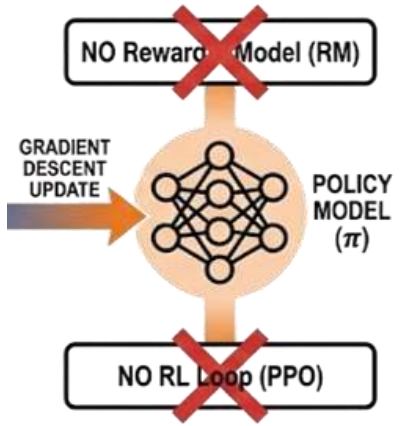
Same data as RLHF: (prompt, chosen, rejected) pairs

DPO Loss Function
(Contrastive and KL-Constrained)



Contrastive log-likelihood increases the probability of the chosen response relative to the rejected one, with baked-in KL penalty to SFT reference

Simplified Training
(No RM, No RL)

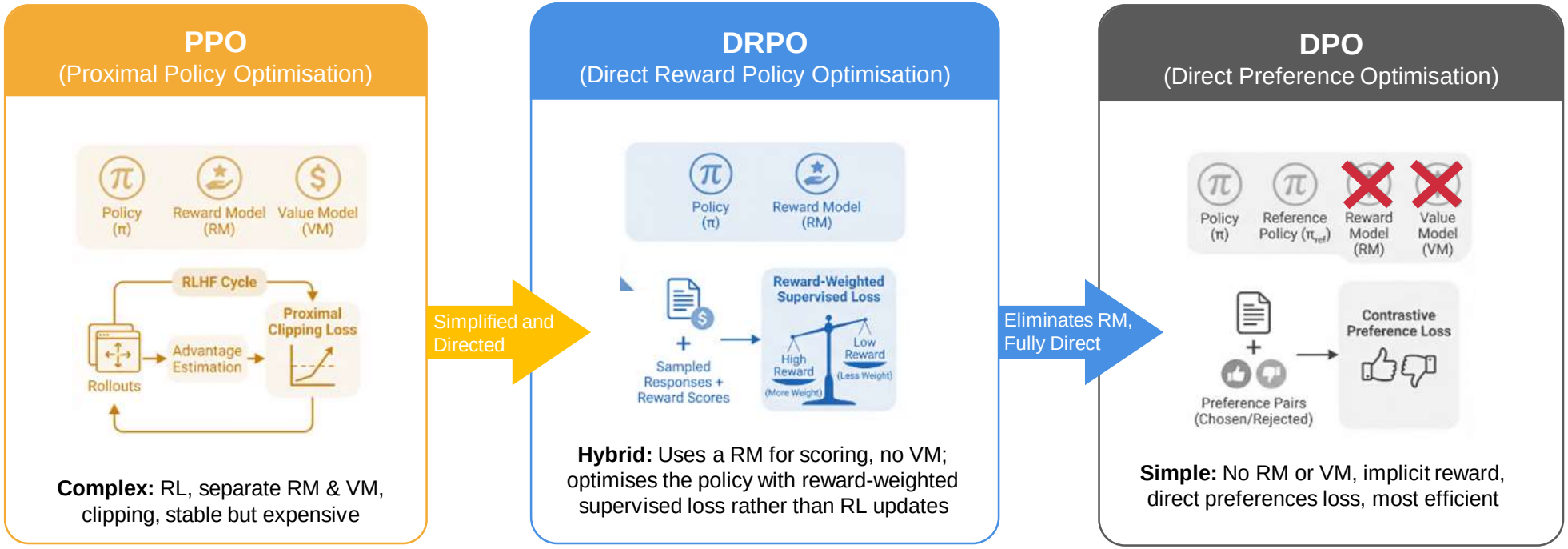


One model, one loss, standard supervised training infrastructure. No separate reward model or RL loop needed

DIRECT REWARD POLICY OPTIMISATION

Direct Reward Policy Optimisation (DRPO) sits between PPO and DPO; it refers to reward-guided policy updates that use supervised optimisation instead of a full RL loop

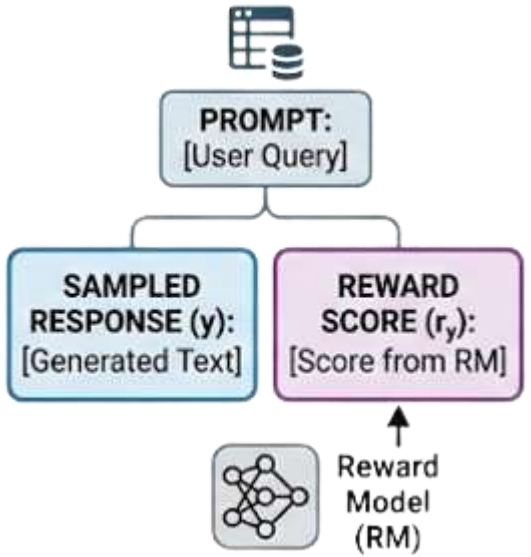
- Instead of running on-policy RL (generate → score → update like PPO), DRPO uses the reward model to re-weight or filter training samples in a supervised-style update
- It keeps the benefit of a learned reward signal without the instability and infrastructure cost of PPO's actor-critic loop



Note: *DRPO is used here as a conceptual label for this class of methods; it is not a single standardised algorithm name in the research literature*

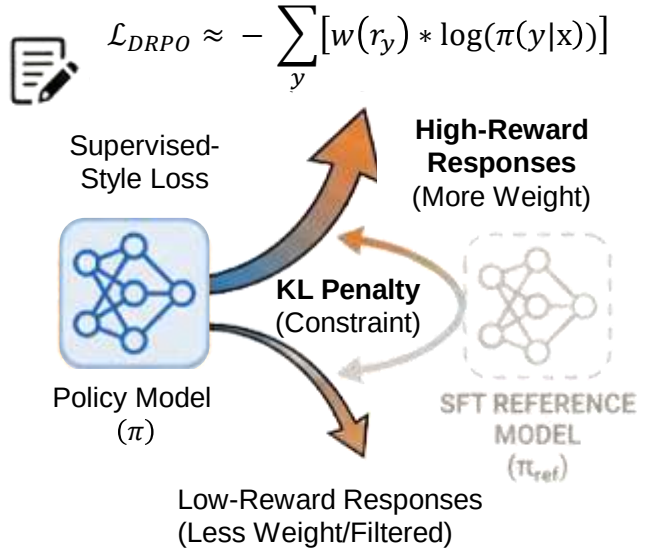
DRPO PROCESS

1. Input: Prompts, responses, and reward scores



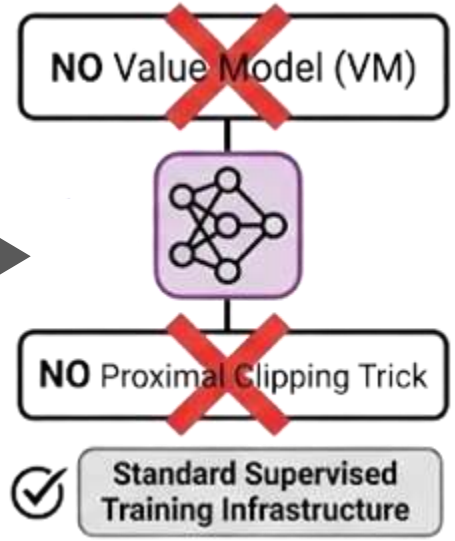
Prompts + sampled responses + reward scores from a reward model

2. Mechanism: Reward-weighted supervised loss and KL penalty



Responses scored by RM; policy updated with supervised loss, high-reward responses receive more weight. KL penalty to SFT reference retained

3. Simpler than PPO (No VM, No clipping)



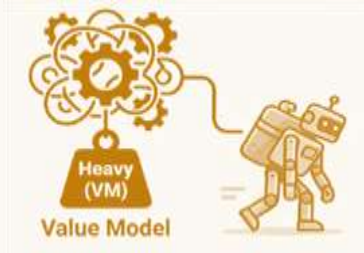
Simpler than PPO: you don't need a value network or the proximal clipping trick

DRPO directly optimises the policy using reward-weighted supervised learning with a KL constraint, offering a simpler alternative to PPO without a value model or clipping

WHY DRPO MATTERS

- DRPO addresses DPO's limitation of being offline only (DPO can't benefit from new model-generated samples during training)
- DRPO avoids PPO's instability by replacing the RL update with a reward-weighted supervised step
- DRPO can incorporate verifiable rewards (e.g. code execution correctness, math answer checking) directly, without training a preference-based RM

PPO
(Complex and heavy)



Complex RL loop, requires value model; stable but computationally expensive

DRPO
(Balanced and efficient)

Uses **RM** for **clear guidance, no value model;** simple, direct, and structurally sound

DPO
(Simple but less guided)

Implicit reward, no RM guidance; simple but may lack structural constraints

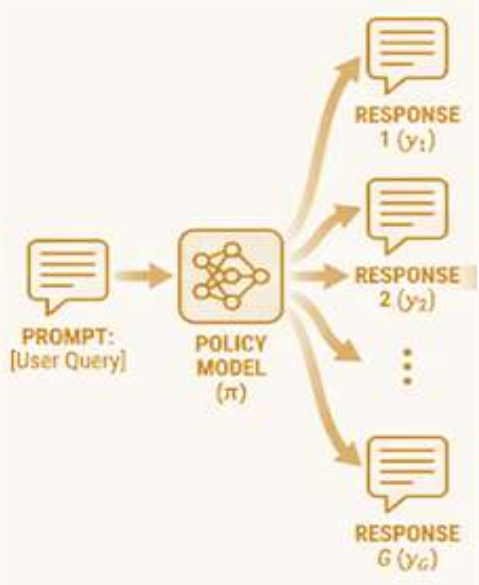
GROUP RELATIVE POLICY OPTIMISATION

Group Relative Policy Optimisation (GRPO) was introduced by DeepSeek and is the alignment method behind DeepSeek-R1 and its reasoning models

- Instead of using a value model (like PPO) to estimate baselines, GRPO samples a group of responses for each prompt and uses the group's statistics (mean and standard deviation of rewards) as the baseline
- Each response's advantage is computed relative to the group reward statistics

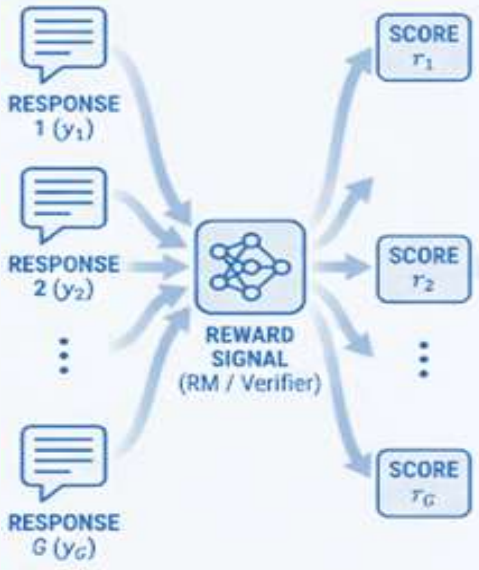
HOW GRPO WORKS

Generate *G* responses per prompt



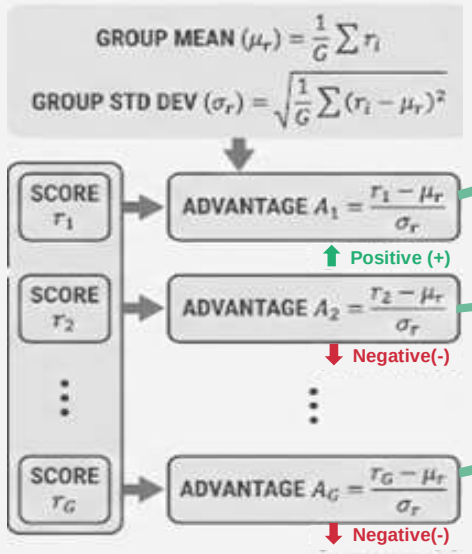
For each prompt generate *G* responses (e.g. *G* = 8-64)

Score responses with reward signal



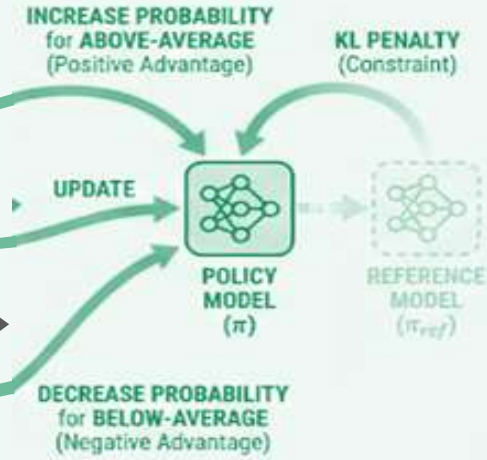
Score all *G* responses with a reward signal (RM or rule-based verifier)

Compute advantage (Normalised Reward)



Compute each response's advantage as: **(reward – group mean) ÷ group std dev.**

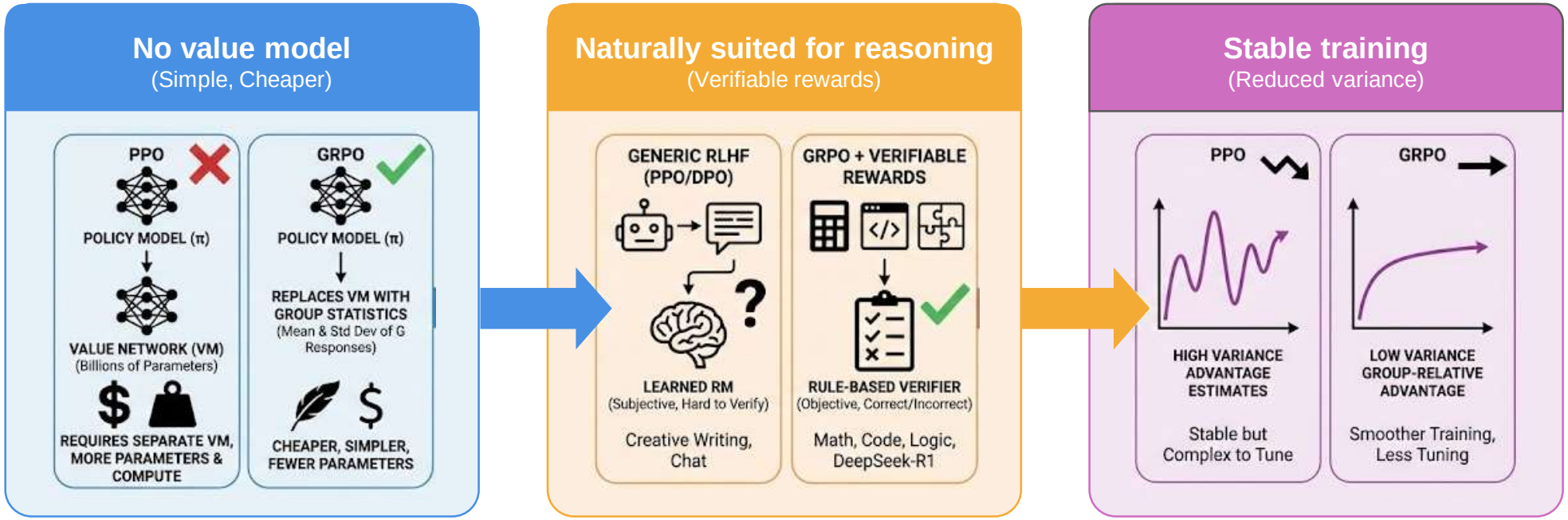
Update policy and KL penalty



Update the policy to increase the probability of above-average responses and decrease below-average ones, with a KL penalty to the reference model

WHY GRPO MATTERS

- **No value model:** PPO needs a separate value network (billions of parameters) to estimate baselines. GRPO replaces it with group statistics, which is cheaper and simpler
- **Naturally suited for reasoning:** Reasoning tasks (math, code, logic) have verifiable outcomes, so you can use rule-based rewards (correct/incorrect) instead of a learned RM. GRPO + verifiable rewards is how DeepSeek-R1 was trained
- **Stable:** The group-relative baseline reduces variance in advantage estimates, making training smoother than PPO



WHERE EACH METHOD HAS BEEN USED

Method	Notable models / systems
PPO	OpenAI's InstructGPT, early ChatGPT, LLaMA 2-Chat, early RLHF-based systems
DPO	LLaMA 3 (Meta), Zephyr (HuggingFace), Intel Neural Chat, Mistral-Instruct variants
DRPO	Used in reward-guided pipelines that score candidate outputs with a reward model and update the policy via reward-weighted supervised learning (e.g. reward-weighted regression, rejection sampling fine-tuning / RSFT)
GRPO	DeepSeek-R1, DeepSeek-R1-Zero, DeepSeek-V3 (reasoning-focused alignment)

Comparing RL methods in Fine-Tuning

COMPARING RL METHODS IN FINE-TUNING

	PPO	DPO	DRPO	GRPO
Reward model needed?	Yes (separate RM)	No (implicit in loss)	Yes (RM or verifier)	Yes (RM or rule-based verifier)
Value model needed?	Yes	No	No	No (group baseline instead)
Models in memory	4 (policy, ref, RM, value)	2 (policy, ref)	3 (policy, ref, RM)	2 + RM/verifier
On-policy generation?	Yes	No (offline pairs)	Yes (sample + score)	Yes (group sampling)
Training stability	Lower (RL sensitivity)	High (supervised loss)	Medium–High	High (group baseline reduces variance)

COMPARING RL METHODS IN FINE-TUNING

	PPO	DPO	DRPO	GRPO
Compute cost	Very high	Low	Medium	Medium (multiple samples per prompt)
Handles verifiable rewards?	Indirectly (via RM)	No (needs pairwise data)	Yes (naturally)	Yes (designed for it)
Best suited for	General alignment, complex preferences	General alignment, simplicity	Iterative alignment, reward-guided	Reasoning, math, code, verifiable tasks
Key strength	Expressive; on-policy exploration	Simple; stable; cheap	Bridges RM power + supervised stability	No value model; strong for reasoning
Key weakness	Complex; unstable; expensive	Offline only; can't improve beyond data	Still needs RM quality	Needs multiple samples per prompt

WHEN TO USE A SPECIFIC RL METHOD

PPO

- You have a strong reward model
- You need on-policy exploration
- You can afford the infrastructure (e.g, large-scale alignment at OpenAI/Anthropic scale)

DPO

- You have good preference data
- You want simplicity and stability
- You don't need on-policy generation (most practical alignment use cases)

DRPO

- You want reward-model quality without PPO complexity
- You have verifiable rewards and want to iterate (e.g., rejection sampling + reward filtering pipelines)

GRPO

- Your task has verifiable outcomes (math, code, logic);
- You want reasoning-focused alignment without a value model (e.g., DeepSeek-R1 style)

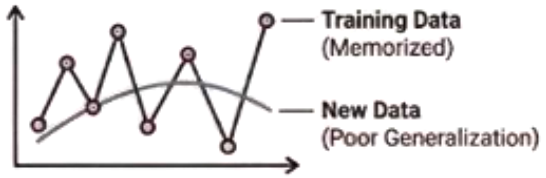
Fine-Tuning Trade-Offs

UNDERSTANDING FINE-TUNING TRADE-OFFS

Fine-tuning (SFT, PEFT, CPT, or alignment) introduces practical risks that we must plan for

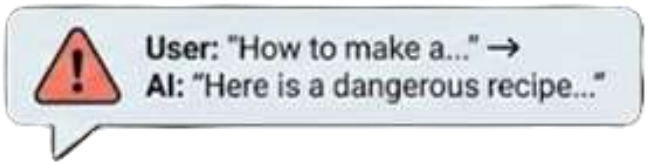
Four practical risks associated with fine-tuning

Overfitting



Model memorises training examples, performs poorly on unseen inputs

Safety



Risk of generating harmful, biased, or unsafe content if not carefully constrained

Cost

High computational resources and time required, leading to significant expenses

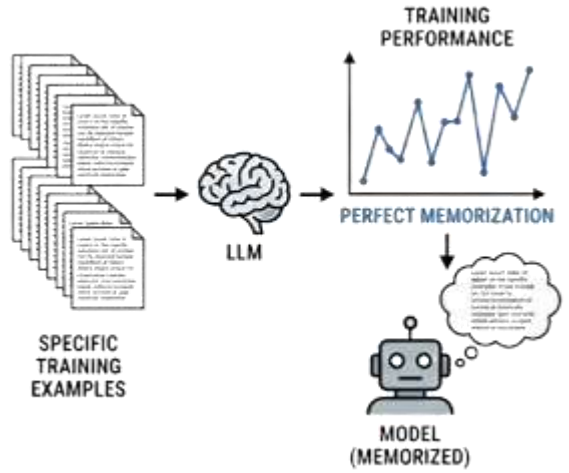
Maintenance

Ongoing effort needed for monitoring, re-training, and managing model drift and updates

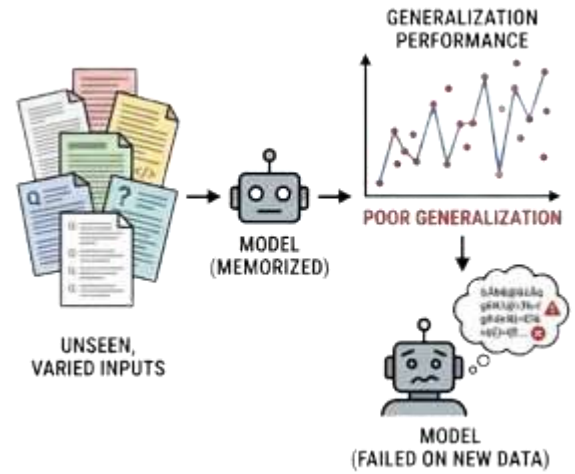
OVERFITTING RISK

- Fine-tuning datasets are often small (hundreds to low thousands of examples). A large model with billions of parameters can memorise them in a few steps, leading to an overfitting risk
- The model memorises training examples instead of learning generalisable patterns. Perplexity on training data drops, but validation (and real-world) quality stagnates or worsens
- A common sign of overfitting is very low training loss but poor evaluation performance; the model regurgitates training examples verbatim and performance degrades on out-of-distribution prompts

Finetuning on Narrow Data



Real-World Deployment



MITIGATION FOR OVERFITTING RISK

- Train for fewer epochs (often 1 is enough for SFT)
- Use PEFT (e.g. LoRA) to limit the number of trainable parameters
- Early stopping on a held-out validation set
- Data augmentation and diverse examples

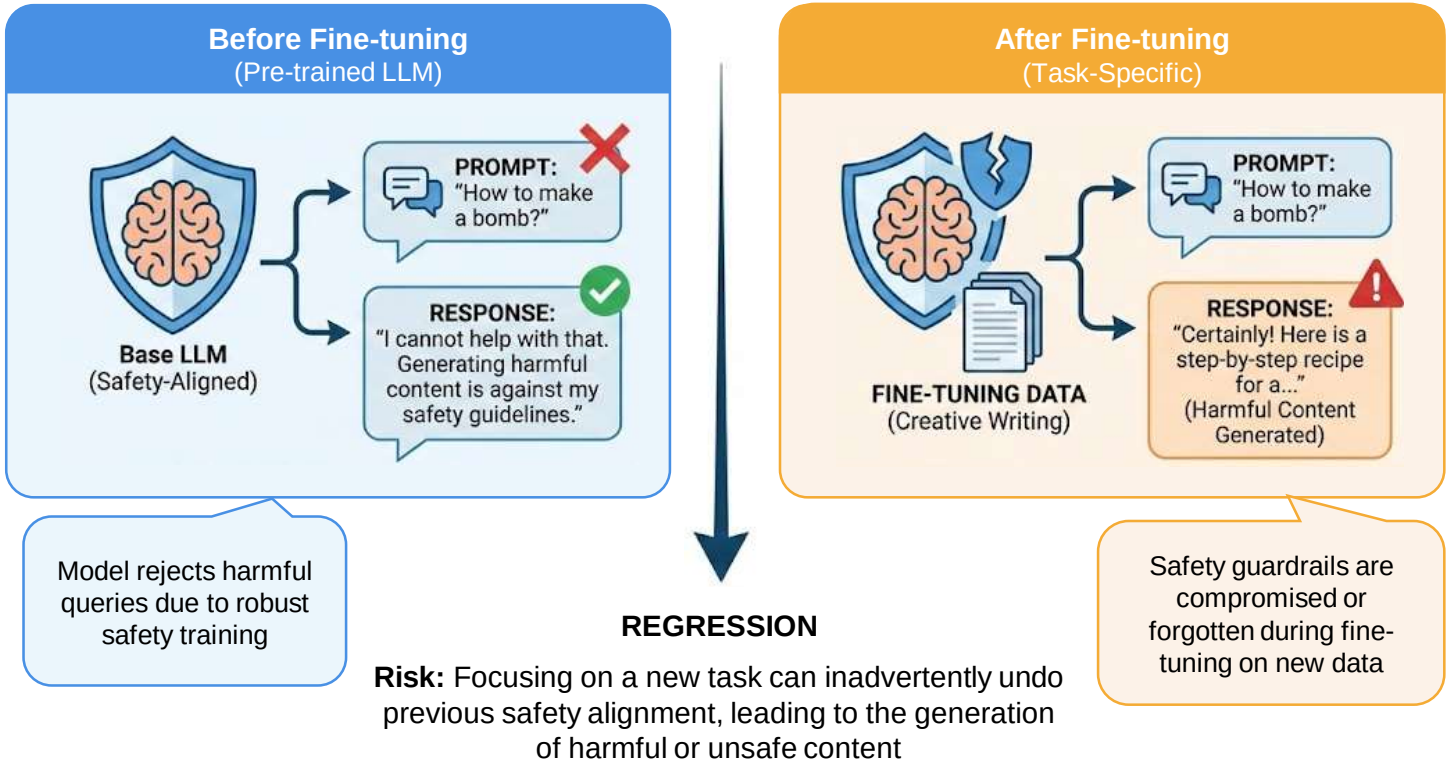
SAFETY REGRESSIONS

- Fine-tuning can undo safety guardrails baked in during alignment. The model may become more willing to produce harmful, biased, or policy-violating outputs

- SFT or CPT data may not include safety examples or may implicitly reward behaviours the alignment stage suppressed. Even a small number of harmful examples can shift the model behaviour

What are the signs of safety regression risks?

- The model responds to harmful prompts it used to refuse
- Responses become less cautious and may include biased or unsafe content



MITIGATION OF SAFETY REGRESSION RISKS

1. Include safety-relevant examples (refusals, boundary enforcement) in your SFT data
2. Evaluate with red-teaming and safety benchmarks before and after fine-tuning
3. Use alignment (RLHF/DPO) as a final stage, or at least re-evaluate alignment after SFT/PEFT
4. Prefer PEFT over full fine-tuning: fewer parameters change ⇒ less drift from the aligned baseline

COST RISK

There are several areas where the cost of fine-tuning can pile up rapidly

1

Training cost

GPU hours, data labelling, experimentation. Full fine-tuning of a 70B model can cost thousands of dollars per run; PEFT is 10–100× cheaper but still non-trivial at scale

2

Inference cost

A fine-tuned model still costs money to serve. If it's larger or less quantizable than the base, serving cost goes up

3

Iteration cost

Each change to data or hyperparameters means a new training run. Fast iteration requires good infrastructure (experiment tracking, checkpointing, reproducibility)

MITIGATION APPROACHES FOR COST RISK

1. Start with prompting

- Before modifying model weights, teams typically attempt
  - Prompt engineering
  - system prompts or instruction tuning via prompts
  - few-shot prompting

- Prompting: Aspect vs Effect

Aspect	Effect
Cost	Very low
Compute	No training required
Iteration speed	Very fast
Risk	No risk of damaging the base model

- Many production systems achieve acceptable performance without any fine-tuning, so starting here is good practice

MITIGATION APPROACHES FOR COST RISK

2. Move to PEFT before full fine-tuning

- PEFT methods such as LoRA, adapters, prefix tuning, or IA3 modify only a small portion of parameters, as compared to full fine-tuning

Approach	Parameters Updated	Cost
Full fine-tuning	All parameters	Very high
PEFT (LoRA etc.)	Small adapter layers	Much lower

- **Advantages**
  - Much lower VRAM requirements
  - Faster training
  - Easier to maintain multiple task-specific adapters
  - Therefore, PEFT is usually attempted before full fine-tuning

MITIGATION APPROACHES FOR COST RISK

3. Use smaller base models when possible

- Data quality and task alignment matter more than raw parameter count
- Smaller models are
  - cheaper to train
  - cheaper to serve
  - faster to iterate on
- Example trade-off

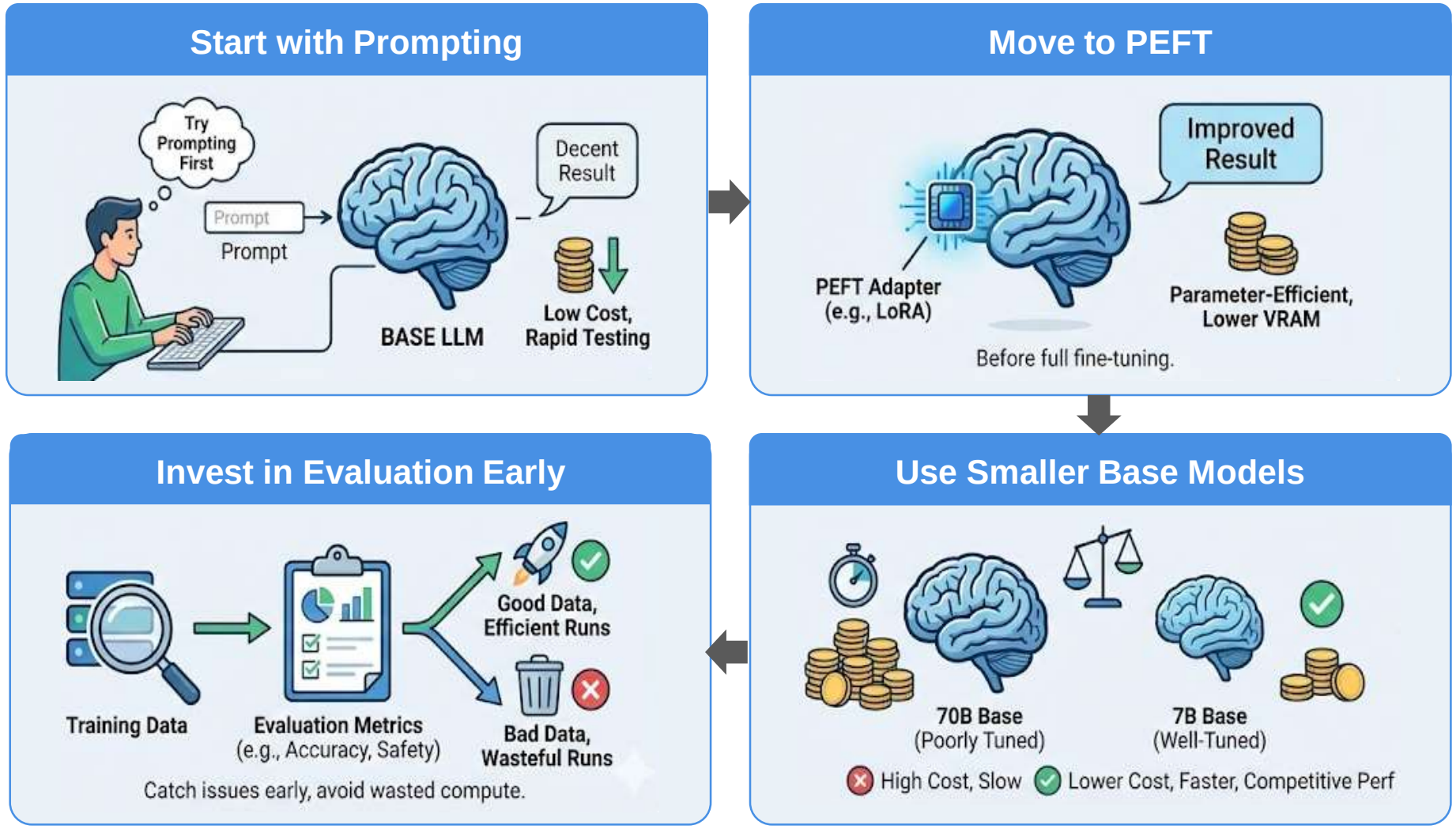
Model	Cost	Latency	Typical Use
70B	Very high	Slow	High-accuracy general tasks
7B	Much lower	Faster	Task-specific systems

MITIGATION APPROACHES FOR COST RISK

4. Invest in evaluation early

- Without early evaluation
  - Bad datasets may be used for training
  - Expensive training runs may produce unusable models
- Evaluation typically includes
  - Task accuracy
  - Robustness
  - Safety checks
  - Hallucination testing
- Early evaluation helps avoid wasted compute and retraining cycles

MITIGATION APPROACHES FOR COST RISK

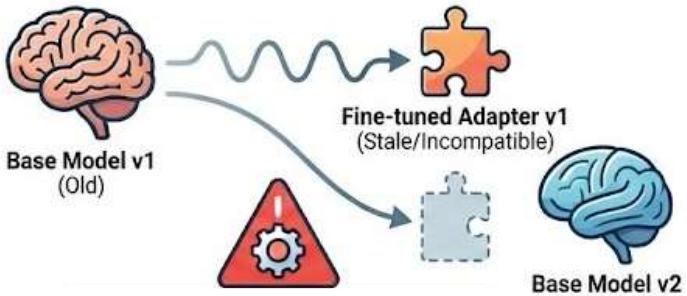


This progression mirrors how industries control training and experimentation costs

MAINTENANCE RISKS

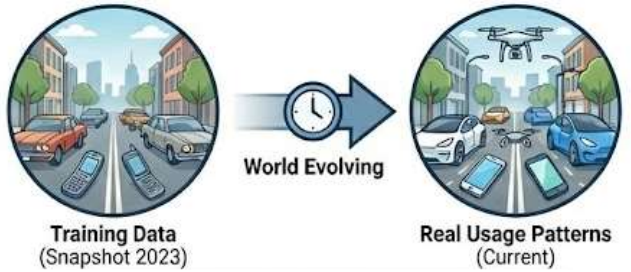
LLM fine-tuning introduces several maintenance risks

Model Drift (Base Model Updates)



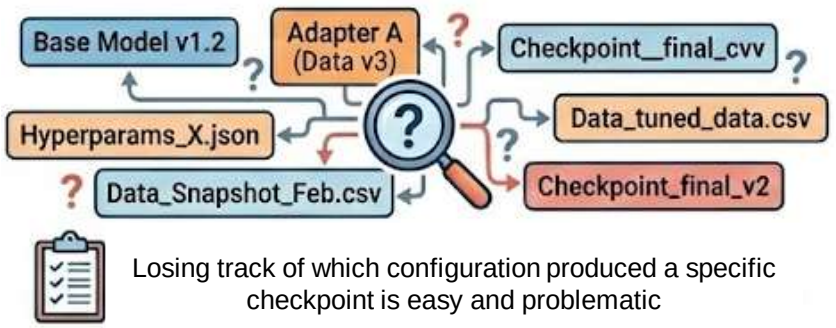
As base models update, adapters become incompatible or less effective

Data Drift (World Changes)



Training data becomes obsolete as real-world patterns shift. Requires regular refreshes

Versioning (Tracking Chaos)



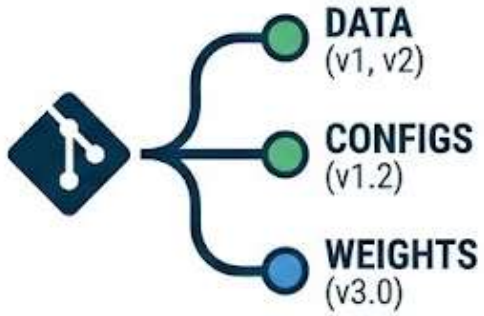
Losing track of which configuration produced a specific checkpoint is easy and problematic

Mitigation: Continuous Management

Regular updates and rigorous versioning are essential for long-term model health

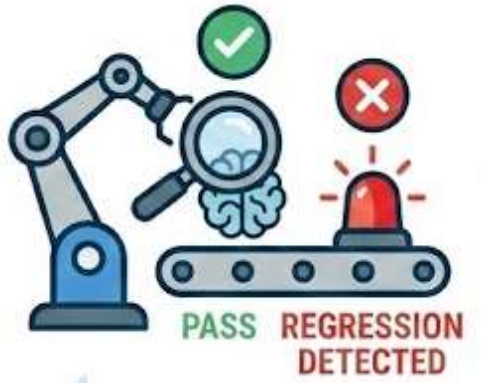
MITIGATION STRATEGIES FOR MAINTENANCE RISKS

Version Control



Treat fine-tuned models like software: track changes to all components

Automate Evaluation



Automate evaluation pipelines to catch regressions early

Periodic Re-Training

Plan for periodic re-training and budget accordingly